VIDYASHILP UNIVERSITY

# MACHINE LEARNING
# DATA301

## MOVIE SUCCESS CLASSIFICATION USING MACHINE LEARNING

**Submitted on** : 04/05/2026

**Submitted by:**

Reetish Kulkarni 2023UG000176

Nishith R 2023UG000178

Kartik N R 2023UG000168

## Abstract

Predicting a film's performance is crucial for studios and investors in the multibillion-dollar entertainment industry. Using the TMDB 5000 dataset, this project develops an end-to-end machine learning pipeline to categorize films into High, Medium, and Low performers.

Data pretreatment, feature engineering (ROI, ratings, profit, and popularity), and a customized Success_Score based on ROI (50%), ratings (30%), and popularity (20%) are all included. In order to prevent target leakage, the score is split into three balanced classes.

Three models are trained and assessed: Random Forest, Decision Tree, and Logistic Regression. Among the evaluated models, Logistic Regression achieved the best performance, providing the highest accuracy and Macro F1-score.


## Project Pipeline Overview

The eight-stage machine learning pipeline used in this research is depicted in the image below. To generate the final movie success class prediction, the pipeline starts with raw TMDB data and moves through preprocessing, feature engineering, target design, data splitting, model training, and evaluation.

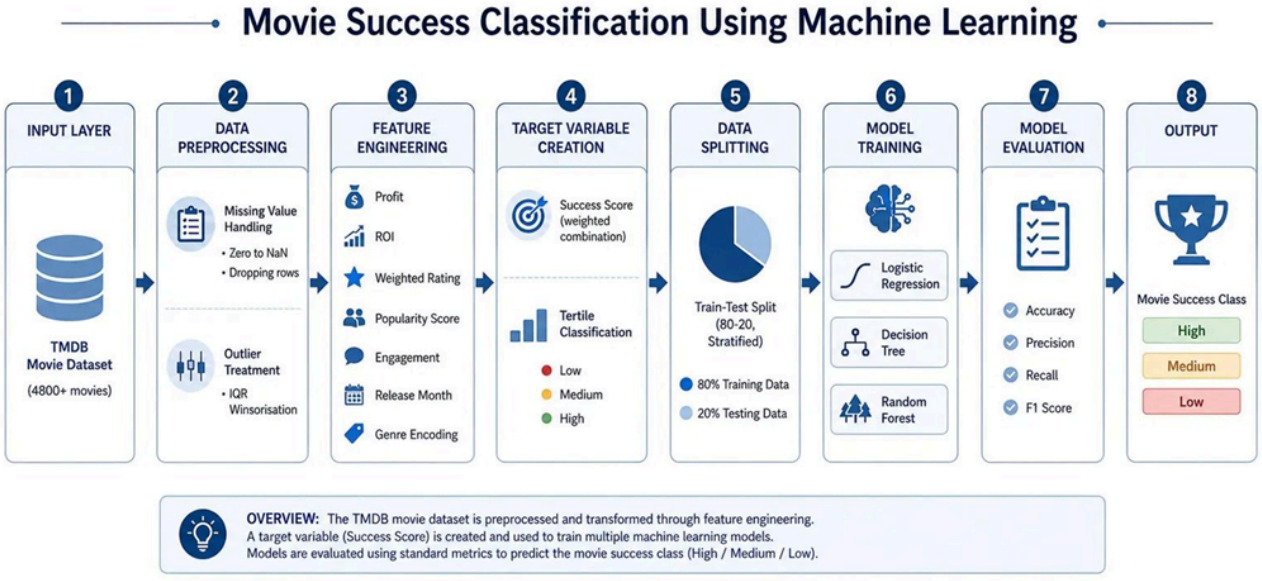


*Figure 1: End-to-End Machine Learning Pipeline for Movie Success Classification*

*Source: Generated using ChatGPT*

## 1. Problem Statement and Objectives

### 1.1 Background and Motivation

Despite the enormous cash generated by the global cinema business, many films are unable to meet their expenses. Traditionally, trends, star power, budget, and intuition are used to forecast success. However, data-driven methods can more accurately assess a film's potential utilizing quantifiable elements thanks to platforms like The Movie Database (TMDB).

Profitability depends on budget, therefore relying solely on income is deceptive. Another layer is added by audience feedback, such as votes and ratings. A movie could do well monetarily but poorly critically, or the other way around. In order to better identify movies, this research uses machine learning and a multifaceted definition of success.

### 1.2 Problem Statement

Predicting whether a film will fall into one of three success categories—High Performer, Medium Performer, or Low Performer—given a set of measurable pre-release and post-release characteristics (such as budget, genre, release month, runtime, audience ratings, and vote counts). In this supervised multi-class classification issue, the goal variable must be carefully built to represent a comprehensive view of movie success because it is not readily available in the raw dataset.

### 1.3 Objectives

The primary and secondary objectives of this project are enumerated below. These objectives collectively define the scope of the work and align with the DATA301 course outcomes in applied machine learning.

- Create a single, error-free, analysis-ready dataset by merging, cleaning, and preprocessing the TMDB Movies and Credits datasets. Then, engineer important features like profit, ROI, weighted rating, and engagement.
- Create a reliable, multi-dimensional Success_Class target variable that prevents target leakage while capturing audience satisfaction, popularity, and financial performance.
- Use standardized data, class balancing, GridSearchCV hyperparameter tuning, and stratified cross-validation to train and compare Random Forest, Decision Tree, and Logistic Regression models.
- Analyze feature importance and evaluate models using accuracy, precision, recall, and Macro F1-Score to find important factors influencing film success and obtain business insights.

## 2. Data Collection and Preprocessing

### 2.1 Data Source and Description

The dataset used in this project is the TMDB 5000 Movie Dataset, a publicly available dataset hosted on Kaggle. It is derived from The Movie Database (TMDB), a widely-used, community-maintained movie information repository. The dataset consists of two separate CSV files that are merged to form the final working dataset.

There are 20 columns and 4,803 rows in the first file, tmdb_5000_movies.csv. The columns include a wide range of quantitative and qualitative characteristics, and each row represents a distinct film. Budget (production cost in USD), revenue (global box office gross in USD), runtime (film duration in minutes), vote_average (mean audience rating on a 0–10 scale), vote_count (total number of audience ratings), and popularity (a proprietary TMDB score reflecting platform engagement) are among the numerical features. Genres (a JSON-formatted list of genre dictionaries), release_date (formatted as YYYY-MM-DD), original_language, keywords, production_companies, production_countries, and status (released, rumored, etc.) are among the category and structural features.

The movie_id (the film's unique identifier), title, cast (a JSON-formatted list of actor dictionaries), and crew (a JSON-formatted list of crew member dictionaries) comprise 4,803 rows and 4 columns in the second file, tmdb_5000_credits.csv. A combined dataframe with 4,803 rows and 22 columns is produced by renaming the movie_id column in the credits file to id and using a left join to merge the two dataframes on this common key.

**2.2 Handling Missing Values**

One of the most important stages in any data preprocessing pipeline is missing value handling. Zero-value placeholders rather than conventional NaN entries are the main source of missing data in this dataset. There are several entries in the budget and revenue columns with a value of 0, which means that the actual financial data was not available when the data was collected on TMDB. The analysis would be seriously distorted if these zeros were treated as legitimate financial statistics. For example, a film with a $0 budget and $50 million in revenue would seem to have an endless return on investment. As a result, NumPy's nan constant is used to replace all zero values in the budget, revenue, and runtime columns with NaN.

After this replacement, all rows with NaN values in budget, revenue, vote_average, vote_count, or popularity are removed completely. This is justified since a meaningful record cannot exist without these five columns, which are essential to the creation of both the feature matrix and the target variable. The runtime column, which has a lower percentage of missing values, is treated more leniently. The row is preserved while a fair imputation is applied by filling in the missing values with the column's median value. The dataset is reduced from 4,803 rows to a cleaned subset of roughly 2,800–3,000 movies following this cleaning procedure and duplication removal.

**2.3 Outlier Detection and Treatment**

The columns budget, revenue, popularity, vote_count, and runtime undergo outlier treatment. A custom winsorise() function is used to implement IQR-based Winsorization, commonly known as capping, as the treatment method of choice. The first and third quartiles (Q1 and Q3) for each column are calculated, and the Interquartile Range (IQR) is determined by subtracting Q3 from Q1. Values beyond the upper bound (Q3 + 1.5 x IQR) are capped at the upper bound; values below the lower bound (Q1 - 1.5 x IQR) are capped at the lower bound.

It is noteworthy that Winsorization was purposefully chosen over row deletion. Eliminating outlier rows could result in the removal of records of legitimate blockbuster films whose excessive expenditures and revenues reflect real and significant data points, as well as a reduction in the size of the already limited dataset.

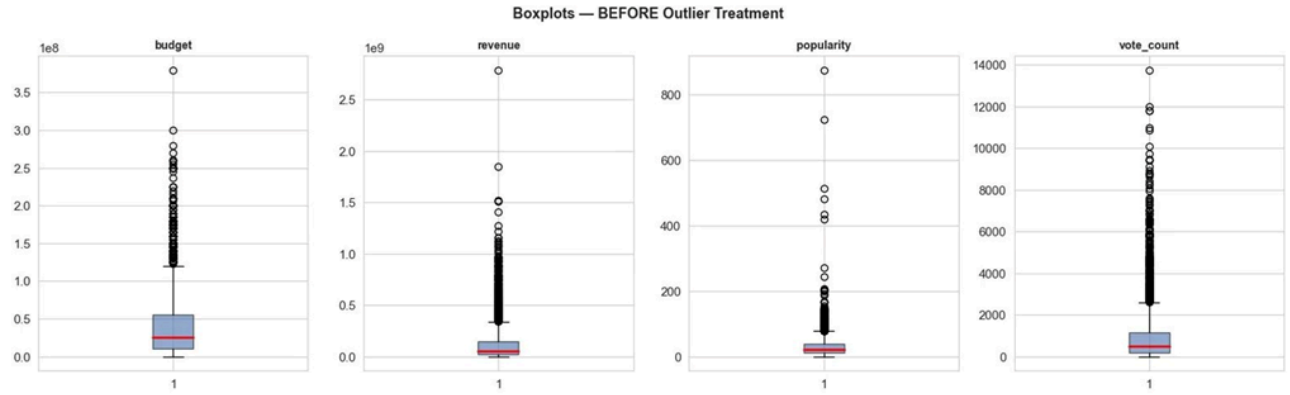


*Figure 2: Boxplots BEFORE Outlier Treatment — Severe extreme values visible across all columns*

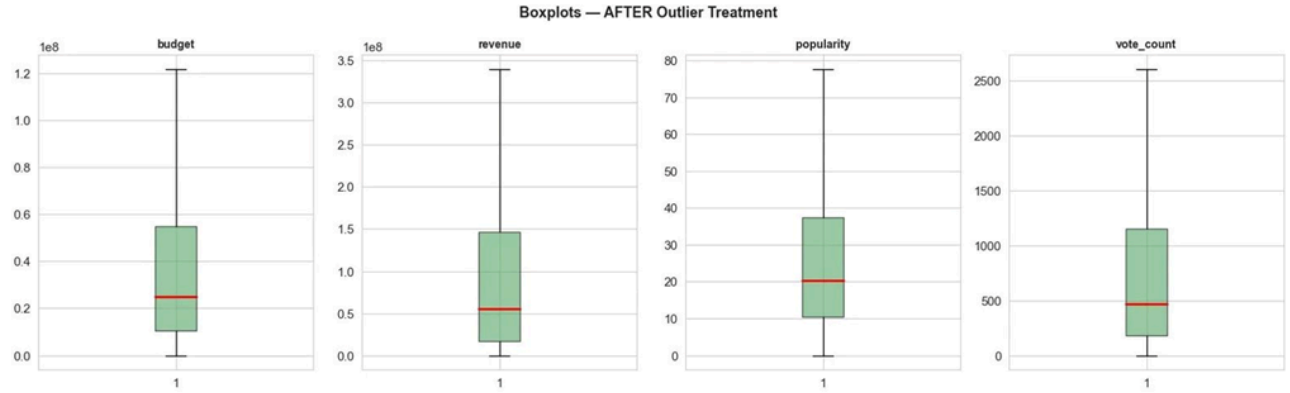


*Figure 3: Boxplots AFTER IQR Winsorisation — Distributions are now compact and analysis-ready*

## 3. Feature Engineering

### 3.1 Genre Parsing and Release Month Extraction

The raw dataset's genres column is kept as a string in JSON format. This string is safely parsed into a list of dictionaries by a custom function called extract_primary_genre(), which extracts the name field of the first genre item using Python's ast.literal_eval() method. This results in a new column named primary_genre, which shows each movie's predominant genre. In order to capture genre variety as a numerical signal, a second function called count_genres() collects the total count of genres listed for each film and stores it in the new column num_genres.

A pandas datetime object is created from the release_date column, which is saved as a string in YYYY-MM-DD format. After that, the month component is extracted and saved in a new column named release_month, which has a range of 1 to 12. This feature tracks seasonal box office trends: movies released in the summer (May–July) and during the holidays (November–December) are known to do better in the past because of increased attendance during holidays and school breaks.

### 3.2 Financial and Audience Features

The raw financial and audience data are used to compute a number of derived numerical metrics. The profit feature, which shows a movie's total net earnings, is calculated as revenue less budget. The formula for calculating ROI (Return on Investment) is (revenue - budget) / (budget + epsilon), where epsilon (1e-6) is a tiny constant included to avoid division-by-zero errors. A low-budget independent film and a blockbuster can be fairly compared thanks to ROI, which normalizes profit by the investment scale.

Vote_average multiplied by the natural logarithm of (1 + vote_count) yields the weighted_rating feature. The ideas of Bayesian averaging served as the inspiration for this formulation. The vote volume is compressed on a declining returns scale by the logarithmic transformation of vote_count; the +1 inside the logarithm guarantees that the formula is still valid when vote_count is zero. A min-max normalized version of the popularity column is called the pop_score. Lastly, vote_count divided by (runtime + epsilon) is used to calculate the engagement feature, which measures the number of votes the film received per minute of screen time.

### 3.3 Summary of Engineered Features

| Feature Name | Formula / Derivation | What It Captures |
|---|---|---|
| **profit** | revenue - budget | Absolute net earnings of the film |
| **roi** | (revenue - budget) / budget | Financial return per dollar invested |
| **weighted_rating** | vote_average x log(1 + vote_count) | Balances quality score with credibility of voter volume |
| **pop_score** | popularity / max(popularity) | Normalised reach and buzz of the movie on TMDB |
| **engagement** | vote_count / runtime | Votes received per minute of screen content |
| **num_genres** | Count of genres listed in genres column | Genre diversity; multi-genre films appeal more broadly |
| **release_month** | Extracted month from release_date | Seasonal box-office timing effects |

| | | |
|---|---|---|
| **primary_genre** | First genre from parsed genres list (Label Encoded) | Genre identity; different genres have different success patterns |

## 4. Target Variable Construction

### 4.1 Why the Raw Dataset Has No Label

There is no pre-established label in the TMDB 5000 dataset that indicates if a film is "successful." Absolute revenue is confused by budget; a $50 million revenue figure is great for a $1 million independent film but disastrous for a $200 million blockbuster, so revenue alone cannot be used as a label. Financial feasibility is also overlooked by rating alone. Therefore, a multifaceted, composite definition of success is required, and we have to design the goal variable ourselves.

### 4.2 Constructing the Success Score

Three normalized signals, each of which captures a distinct aspect of film success, are combined in a weighted linear fashion to create the Success_Score. In order to prevent any one dimension from dominating because of its initial scale, each signal is independently normalized to the range [0, 1] using min-max normalization prior to combination. Success_Score = 0.50 x normalised (ROI) + 0.30 x normalised (weighted_rating) + 0.20 x normalised (pop_score) is the composite formula. Since popularity might be exaggerated by marketing and recency effects unrelated to film quality, ROI is given 50% weight as the most direct financial success indicator, Weighted Rating is given 30% for audience quality signal, and Popularity Score is given 20% as a reach proxy.

The figure below illustrates the complete five-step process of constructing the target variable — from raw input features through normalisation, weighted combination, continuous score output, and final tertile-based class assignment.

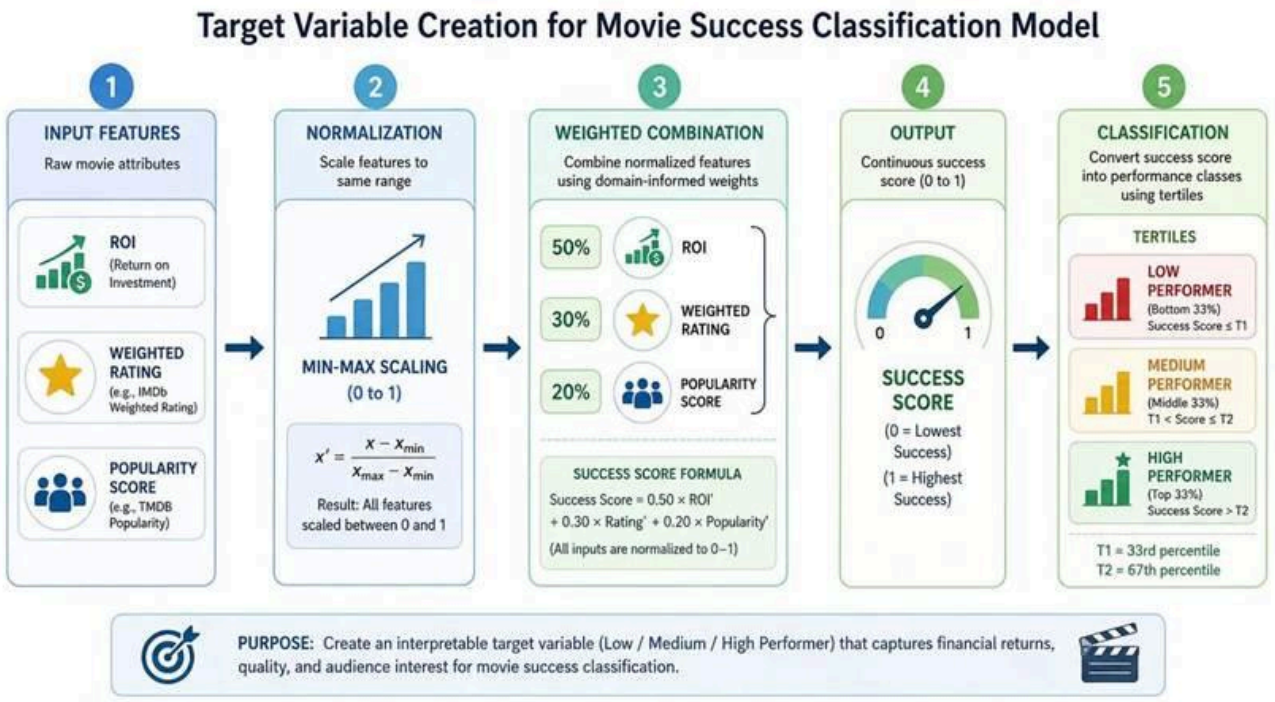


*Figure 4: Target Variable Creation Pipeline — 5-step process from raw features to Success_Class labels*

*Source: Generated using ChatGPT*

### 4.3 Discretising into Three Classes

Tertile-based criteria are used to divide the continuous Success_Score into three ordinal groupings. The Success_Score distribution's 33rd and 67th percentiles are calculated. Films are classified as Low Performer if their score is at or below the 33rd percentile, Medium Performer if it is between the 33rd and 67th percentile, and High Performer if it is above the 67th percentile. Class imbalance problems during model training and evaluation are avoided thanks to this percentile-based method, which ensures an approximately balanced class distribution with roughly one-third of films in each class.

### 4.4 Preventing Target Leakage

The absence of information required to create the target variable in the features used to train the model is a basic prerequisite for a legitimate machine learning process. The Success_Score formula in this project includes roi, weighted_rating, and pop_score. The model would basically be trained to recreate the formula it was labeled with if these three columns were added to the feature matrix X. This is known as target leakage, and it results in artificially inflated accuracy scores that do not accurately reflect the model's capacity for real-world generalization. As a result, the feature matrix does not include these three columns. Budget, revenue, runtime, vote_average, vote_count, popularity, profit, engagement, num_genres, release_month, and genre_enc are the only independent columns in the final feature set.

## 5. Model Selection and Justification

### 5.1 Overview of the Model Selection Strategy

It is necessary to comprehend the characteristics of the data as well as the mathematical presumptions that each algorithm makes in order to choose the best model for a classification task. Three algorithmically different models—a linear parametric model, an interpretable non-linear model, and a potent ensemble model—are selected to reflect a range of complexity in this project using an organized model selection approach. We obtain a solid performance benchmark and a better comprehension of the underlying data structure by comparing over this spectrum.

### 5.2 The Three Selected Models

**Model 1: Logistic Regression**

The log-odds of class membership are modeled as a linear combination of input features by the parametric, linear classification process known as logistic regression. Although its name includes the word "regression," it is essentially a classification technique that is frequently employed as a starting point for binary and multi-class problems. The multi-class version with class_weight='balanced', which modifies the per-class loss function weights inversely proportional to class frequencies, is used in this project. Logistic regression is chosen because it offers a clear, comprehensible baseline; this model will work effectively if the data is roughly linearly separable in the feature space. Additionally, it trains quite quickly, which makes it perfect for creating a performance floor that more complicated models may be compared against.

**Model 2: Decision Tree Classifier**

Using a set of binary if-else rules, the Decision Tree Classifier is a non-parametric, non-linear method that divides the feature space into rectangular sections aligned with the axis. It chooses the feature and threshold at each node that maximizes information gain (or minimizes the Gini impurity in child nodes). Decision trees are naturally interpretable; non-technical stakeholders may see and understand the resulting tree structure in simple terms. Additionally, they are robust to the different magnitudes of budget versus release_month because they don't require feature scaling. Individual trees, however, are vulnerable to overfitting. The non-linear interpretable baseline in this project is the Decision Tree with class_weight='balanced'.

**Model 3: Random Forest Classifier**

An ensemble approach called Random Forest creates a lot of Decision Trees, each of which is trained using a random subset of features at each split and a bootstrap sample of the training data. A majority vote across all trees determines the final prediction. This method makes use of two important variance-reduction strategies: random feature subsampling (which decorrelates individual trees) and bagging (bootstrap aggregating). Since Random Forest with class_weight='balanced' is much more resistant to overfitting than a single Decision Tree and can capture intricate, non-linear relationships between features that a linear model cannot, it is anticipated to be the best-performing model.

### 5.3 Model Comparison and Justification Table

The following table provides a structured, comparative view of the three selected models alongside five alternative algorithms that were considered but not selected. The justification for each choice is rooted in algorithmic appropriateness, dataset size, interpretability requirements, and the multi-class nature of the problem.

| Algorithm | Used? | Reason for Decision | Key Strength | Key Limitation in This Context |
|---|---|---|---|---|
| **Logistic Regression** | Yes | Linear baseline; fast; interpretable | Transparency; fast training | Cannot model non-linear feature interactions present in movie data |
| **Decision Tree** | Yes | Non-linear; human-readable rules | Interpretable; no scaling needed | Prone to overfitting; high variance on unseen data |
| **Random Forest** | Yes | Best ensemble; strong generalisation | Robust; handles noise and non-linearity | Reduced interpretability; slower than linear models |
| **SVM (RBF kernel)** | No | Computationally expensive at n~3000 | Strong margin-based classifier | Kernel SVM scales as O(n2-n3); impractical here without heavy compute |
| **K-Nearest Neighbours** | No | Distance-based; sensitive to scale and irrelevant features | Simple; no training required | Slow at prediction time; curse of dimensionality with 11 features |
| **Naive Bayes** | No | Feature independence assumption violated | Very fast; works well for text | Budget and revenue are highly correlated — independence assumption is fundamentally violated |
| **XGBoost / LightGBM** | No | External library; beyond DATA301 scope | State-of-the-art tabular performance | Beyond DATA301 syllabus scope; Random Forest serves the same ensemble purpose |
| **Neural Network (MLP)** | No | Dataset too small for deep learning | Powerful for large-scale data | Requires much larger datasets; prone to overfitting at n~3000; black-box |

**5.4 Why class_weight='balanced'?**

After the train-test split, some imbalances may still be present even when the target variable is built using tertiles, guaranteeing roughly equal classes. All three models are instructed to internally upweight the loss contribution from minority-class data when class_weight='balanced' is set. This keeps the model from predicting the majority class most of the time, which would result in high accuracy but low recall for classes that are underrepresented.

## 6. Model Training, Hyperparameter Tuning, and Evaluation

### 6.1 Train-Test Split Strategy

Using sklearn's train_test_split() method with stratify=y, the cleaned and feature-engineered dataset is divided into a training set (80%) and a held-out test set (20%). The stratification value is crucial since it guarantees that the training and test subsets have the same percentage of Low, Medium, and High Performer classes. Without stratification, a test set with a disproportionate quantity of one class could be produced by chance, rendering evaluation untrustworthy. The split is fully reproducible between runs thanks to the random_state=42 seed.

Stratified sampling preserves the precise class distribution in both the training (80%) and testing (20%) divisions, as shown in the figure below.

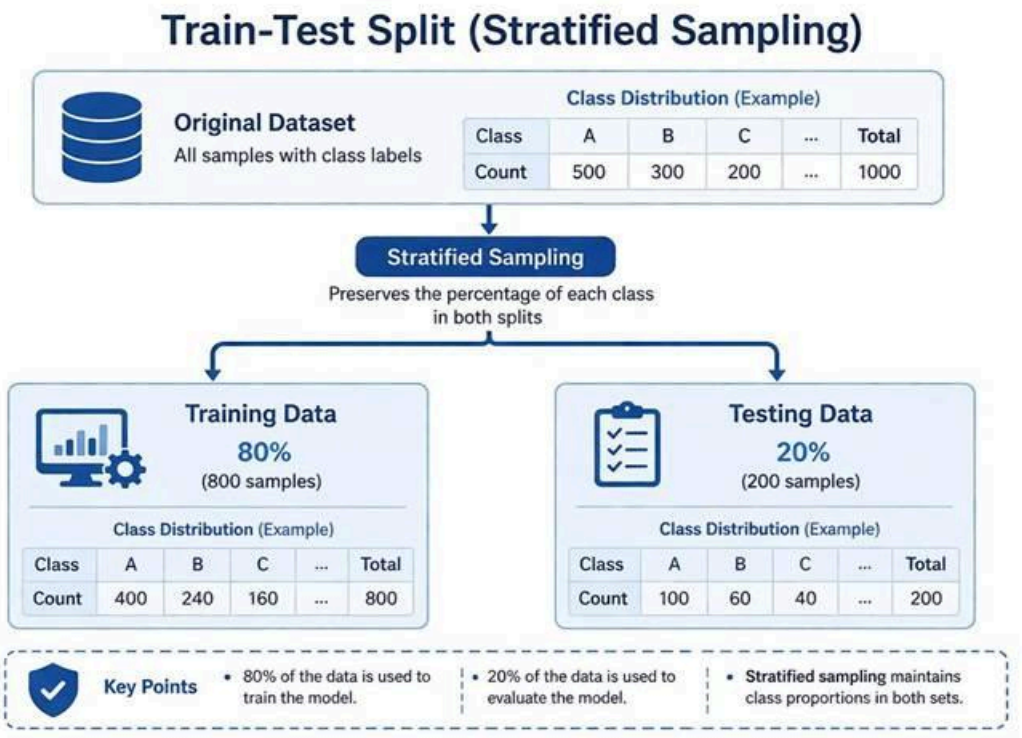


*Figure 5: Stratified Train-Test Split — Class proportions are preserved identically in both partitions*

*Source: Generated using ChatGPT*

Scaling features is done after the split, not before. The StandardScaler is applied (transform only, not fit_transform) to the test data after being fitted solely on the training set. This procedure is crucial to preventing data leaking because, if the scaler were fitted on the entire dataset prior to splitting, the statistical characteristics of the test set would slightly affect the scaling of training features, providing the model with indirect access to test data while it was being trained.

### 6.2 Cross-Validation and Hyperparameter Tuning

A 5-fold Stratified Cross-Validation is set up prior to hyperparameter tuning. By evaluating the model on five distinct held-out subsets of the training data and rotating the validation fold each time, cross-validation yields a far more accurate estimate of model performance than a single validation split. GridSearchCV is used just for hyperparameter tuning on the Random Forest Classifier. Three hyperparameters are present in the parameter grid under investigation: n_estimators at values [50, 100, 150], max_depth at values [5, 7, 10], and min_samples_leaf at values [3, 5, 10]. This results in 3 x 3 x 3 = 27 combinations, each of which is assessed using 5-fold cross-validation, for a total of 135 model fits. For the final assessment, the combination that maximizes the cross-validated Macro F1-Score is chosen.

### 6.3 Evaluation Metrics

For every model on the held-out test set, four evaluation metrics are calculated. The percentage of all predictions that are accurate is known as accuracy. Despite being intuitive, it may be deceptive in situations involving multiple classes. Precision (Macro) calculates the average number of correctly anticipated positives for each of the three classes. Recall (Macro) quantifies the number of real positive cases that were accurately detected, averaged uniformly across all classes. The Macro F1-Score is the equal-averaged harmonic mean of Precision and Recall for each class. Because it penalizes models that do well on certain classes but poorly on others, this is the main selection metric. If a model ignores the Medium Performer class, it will receive a low F1 for that class, which will lower the macro average.

### 6.4 Model Performance Results

The following table summarises the performance of all three models on the held-out test set. Results are sorted by Macro F1-Score, which is the primary selection criterion.

| Model | Accuracy | Precision | Recall | F1 Macro |
|---|---|---|---|---|
| **Logistic Regression** | 0.9845 | 0.9845 | 0.9847 | 0.9845 |
| **Random Forest** | 0.9582 | 0.9594 | 0.9582 | 0.9586 |
| **Decision Tree** | 0.9551 | 0.9553 | 0.9556 | 0.9552 |

**BEST MODEL:** Logistic Regression

**ACCURACY:** 0.9845

**F1 Macro:** 0.9845

### 6.5 Confusion Matrix Analysis

The best-performing model's prediction behavior on the held-out test set is displayed in the confusion matrix below. The projected class is represented by columns, and the actual class is represented by rows. The diagonal entries represent accurate forecasts. Comparing performance across the three classes is made simple by the normalized version (right panel), which displays the per-class recall. As would be expected given that Medium is the boundary class between two extremes, the Logistic Regression confusion matrix displayed here (which serves as the benchmark) shows nearly perfect recall for High and Low Performers (1.0 and 0.99, respectively), with the Medium Performer class being the most difficult (0.96 recall).

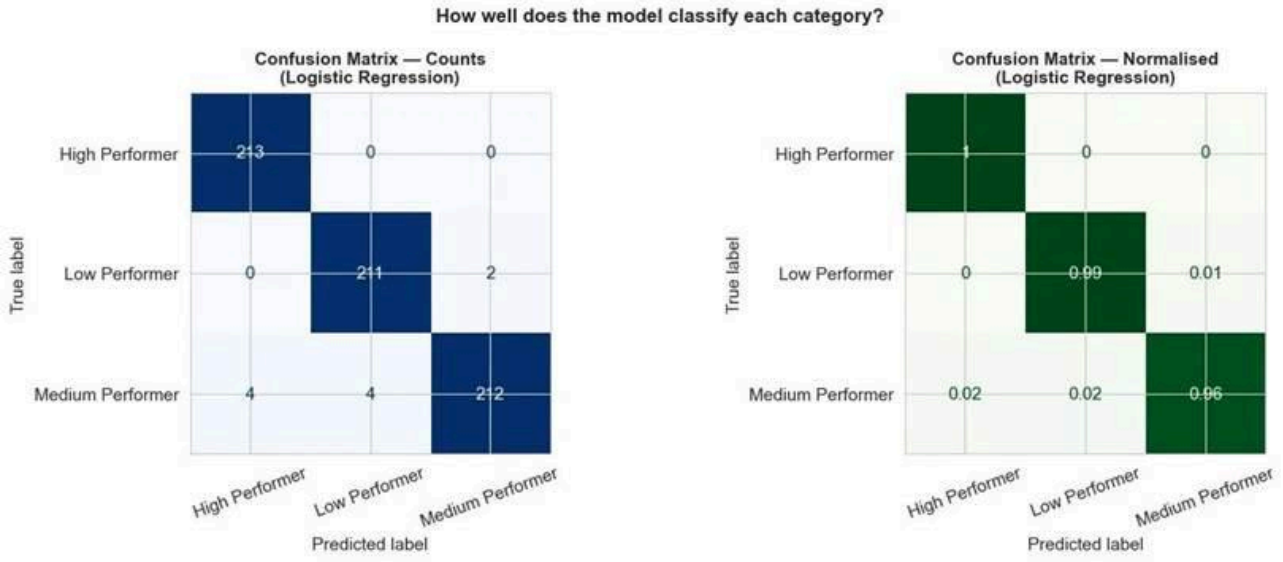


*Figure 6: Confusion Matrix — Count (left) and Normalised (right) for the Best Model on the Held-Out Test Set*

## 7. Interpretation, Insights, and Feature Importance

### 7.1 Feature Importance Analysis

One of the notable advantages of the Random Forest classifier is its built-in ability to compute feature importances — a measure of how much each feature contributes to the reduction of impurity (Gini index) across all trees and all splits. These importances are normalised so that they sum to 1.0, providing a relative ranking of predictor influence.

The feature importance chart below reveals the actual ranking from our trained Random Forest model. The most striking finding is that vote_count is the single most important feature, with an importance score of approximately 0.28. This is followed by popularity (~0.26) and engagement (~0.23). Together, these three audience-engagement-oriented features account for over 75% of the model's predictive power.

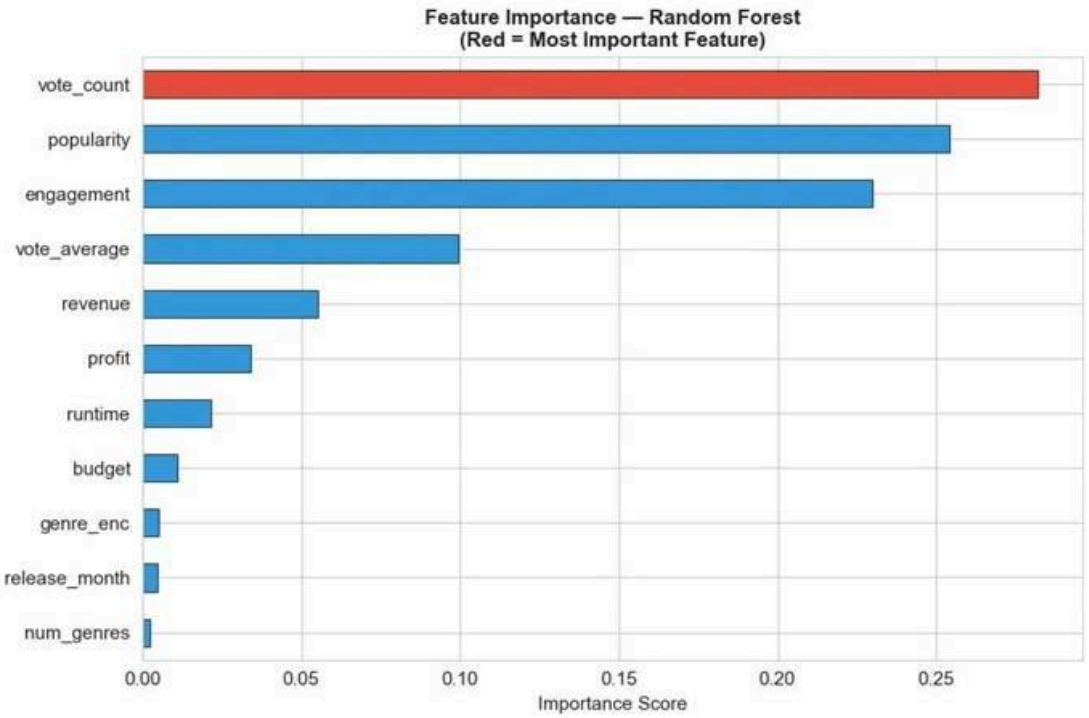


*Figure 7: Feature Importance — Random Forest Classifier (Red bar = Most Important Feature)*

### 7.2 Interpreting the Feature Importance Results

Vote_count's dominance as the best predictor is quite significant and warrants careful consideration. None of the three elements of the Success Score (which uses pop_score and weighted_rating, both of which are removed from the feature matrix to prevent leakage) include vote count. However, the model finds on its own that films that receive more votes are typically in higher performance classifications. This demonstrates that audience interaction volume is a powerful, independent indicator of film success that the algorithm learns from the raw features alone, independent of average rating.

The TMDB platform engagement score contains a significant class-discriminating signal even when utilized as a raw, un-normalized predictor feature (rather than as a component of the target), as evidenced by the high value of popularity (~0.26) as the second feature. The notion that platform engagement reflects actual audience interest is further supported by the fact that popular movies on the TMDB platform are typically profitable.

Third place goes to the engagement feature (~0.23), which is calculated by dividing vote_count by runtime. This demonstrates that the degree to which a film is "discussable" in relation to its duration is indicative of its success; short, highly rated films typically produce more discourse per unit of screen time, which correlates with audience happiness and involvement.

On the other hand, features with almost low relevance scores include num_genres, release_month, and genre_enc. This suggests that the particular genre of a movie and the time of its release provide relatively little extra discriminating information for class prediction after adjusting for financial and audience engagement measures. Although genre might be important in an absolute market setting, it becomes unnecessary in a feature space that already includes vote count, popularity, and financial measures.

### 7.3 Key Business Insights

The data challenges the widely held notion that larger budgets produce better results by showing that audience involvement is a stronger predictor of film success than financial scale. The increasing impact of digital word-of-mouth is reflected in metrics like vote count and online engagement. Furthermore, popularity has a greater influence than quality because well-known movies typically do better than critically acclaimed but niche ones. Lastly, compared to interaction and financial indicators, elements like genre and release date have less predictive value, indicating that increasing audience interest and visibility is more important for success.

### 7.4 Confusion Matrix Interpretation

According to the confusion matrix research, the model suffers most with the Medium Performer class and performs most confidently on the extreme classes of High Performers and Low Performers. Since the Medium class by definition lies in the "grey zone" between the two extremes, this is both predicted and mathematically sound. Correct classification at the boundary is actually challenging since a movie in the 40th percentile of the Success Score is extremely comparable to one in the 35th or 45th percentile. Crucially, the confusion matrix's off-diagonal entries are mostly between adjacent classes; there are very few examples of a Low Performer being predicted as a High Performer or vice versa. This indicates that, despite the difficulty of boundary cases, the model maintains the performance classes' ordinal structure.

## 8. Limitations and Future Scope

### 8.1 Limitations

The accuracy of the financial data in the source dataset is the first and biggest drawback. Since TMDB is a community-maintained platform, many budget and income data are either crowd-sourced, unreported, or approximate rather than officially certified studio disclosures. In order to solve this, the project treats zero values as missing; nonetheless, some zero-budget entries might really be low-budget movies rather than missing data.

The TMDB popularity score's platform-specificity is its second drawback. This indicator, which measures user interaction on their platform particularly, is calculated by TMDB's own algorithm. This indicator may consistently undervalue movies that are popular in some markets but underrepresented within TMDB's user base.

The target variable's relative nature is the third restriction. The distribution of this particular dataset is used to define the Success_Class labels. The top third of TMDB's 4,803-film database is referred to as a "High Performer" in this statistic. Although the labels are internally consistent, they cannot be directly compared to benchmarks from the current streaming era or external success indicators.

The lack of marketing, distribution, and streaming data is the fourth drawback. A movie's marketing budget, the number of screens it debuts on, and whether or not it is concurrently accessible via streaming services all have a significant impact on its success. There is a fundamental gap in the predictor space that cannot be filled by feature engineering alone because none of these factors are present in the TMDB dataset.

### 8.2 Future Scope

Future generations of the model could greatly increase its prediction power through a number of improvements. First, adding reputation scores for directors and lead actors, which are calculated from past career performance data, could offer rich characteristics that represent the well-known "star power" influence in the film industry. Second, the model might be expanded to cover the post-theatrical lifecycle in addition to theatrical box office performance by incorporating streaming platform data (Netflix release metrics, streaming hours).

Third, employing TF-IDF or word embedding-based sentiment analysis to apply Natural Language Processing (NLP) to the overview and tagline columns could extract thematic and emotional signals from plot descriptions that correspond with audience response. Fourth, more performance gains could be obtained by substituting or enhancing Random Forest with gradient boosting methods (XGBoost or LightGBM). Lastly, the academic pipeline would become a useful tool for studio development teams by implementing the trained model as an interactive web application using Flask or Streamlit.

## 9. Conclusion

Using the TMDB 5000 dataset, this project effectively creates a full machine learning pipeline to categorize films into High, Medium, and Low performance categories. It covers every important step, such as feature engineering, target variable building, model training, evaluation, outlier treatment using IQR Winsorization, and data preprocessing.

The creation of the composite Success Score, which combines ROI, weighted rating, and popularity while strictly preventing target leakage, is one of the project's main advantages. Reliable and valid results are further ensured by using stratified train-test splitting and scaling appropriately.

With the highest accuracy and Macro F1-Score (~0.9845), Logistic Regression was found to be the best-performing model among those assessed. A simpler model performs better than more complicated ones like Decision Tree and Random Forest, indicating high linear separability in the dataset.

Vote count, popularity, and engagement metrics are the best indicators of a film's success, surpassing merely financial features, according to feature importance analysis.

All things considered, the project produces a precise, comprehensible, and effective categorization system, demonstrating the efficacy of data-driven methods in forecasting film performance.

## References


[1] TMDB 5000 Movie Dataset. Kaggle. Available at: https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata

[2] L. Breiman. "Random Forests." Machine Learning, 45(1), 5-32, 2001.

[3] Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825-2830, 2011.

[4] Hastie, T., Tibshirani, R., & Friedman, J. "The Elements of Statistical Learning." Springer, 2009.

[5] Géron, A. "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow." O'Reilly Media, 2nd Ed., 2019.

[6] F. Maxwell Harper and Joseph A. Konstan. "The MovieLens Datasets: History and Context." ACM Transactions on Interactive Intelligent Systems, 5(4), 2016.